Combine

Quickstart: Swap tokens across chains

Copy page

Use App Kit to swap tokens on the source chain and bridge to the destination

This quickstart walks you through how to use App Kit to swap tokens across blockchains. The examples show how to swap 1.00 EURC for USDC on Arc Testnet and then bridge that USDC to Ethereum Sepolia, but you can use other **supported tokens or blockchains**.

Prerequisites

Before you begin, ensure that you've:

- Installed **Node.js v22+**.
- Created an EVM wallet using a wallet provider such as **MetaMask** and added **Arc Testnet** and **Ethereum Sepolia**.
- Funded your EVM wallet with testnet tokens:
  - EURC on Arc Testnet (for the swap into USDC)
  - Native tokens on Arc Testnet and Ethereum Sepolia for fees
- Obtained a (free) kit key from **Circle Console**.

Step 1. Set up the project

This step shows you how to prepare your project and environment.

1.1. Set up your development environment

Create a new directory and install App Kit and its dependencies:

Shell

```
# Set up your directory and initialize a Node.js project
mkdir app-kit-quickstart-swap-crosschain
cd app-kit-quickstart-swap-crosschain
npm init -y

# Install App Kit and tools
npm install @circle-fin/app-kit @circle-fin/adapter-viem-v2 viem typescript ts
```

1.2. Configure TypeScript (optional)

> This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

Shell

```
npx tsc --init
```

Then, update the `tsconfig.json` file:

Shell

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Configure environment variables

Create an `.env` file in the project directory:

Shell

```
touch .env
```

Then, add your credentials. Replace `YOUR_PRIVATE_KEY` with the private key for your wallet and `YOUR_KIT_KEY` with the kit key from the Circle Console:

.env

```
PRIVATE_KEY=YOUR_PRIVATE_KEY
KIT_KEY=YOUR_KIT_KEY
```

Edit `.env` files in your IDE or editor so credentials are not leaked to your shell history.

Step 2. Swap and bridge tokens

This step shows you how to set up your script, swap EURC for USDC on Arc Testnet, bridge USDC to Ethereum Sepolia, and check the result.

2.1. Create the script

Create an `index.ts` file in the project directory and add the following code. This code swaps EURC for USDC on Arc Testnet, then bridges that USDC to Ethereum Sepolia:

Using other **tokens** or **chains**? Change the `chain` values in `kit.swap()` and `kit.bridge()`, and the `tokenIn` and `tokenOut` values in `kit.swap()`. The source chain must support Swap and both chains must support Bridge. Bridge transfers USDC only.

TypeScript

```typescript
import { AppKit } from "@circle-fin/app-kit";
import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
import { inspect } from "util";

const kit = new AppKit();

const swapAndBridge = async (): Promise<void> => {
  try {
    const adapter = createViemAdapterFromPrivateKey({
      privateKey: process.env.PRIVATE_KEY as string,
    });

    console.log(
      "---------------Step 1: Swapping EURC for USDC on Arc Testnet-----------
    );
    const swapResult = await kit.swap({
      from: { adapter, chain: "Arc_Testnet" },
      tokenIn: "EURC",
      tokenOut: "USDC",
      amountIn: "1.00",
      config: {
        kitKey: process.env.KIT_KEY as string,
      },
    });

    console.log("Swap result:", inspect(swapResult, false, null, true));

    console.log(
      "---------------Step 2: Bridging USDC to Ethereum Sepolia---------------
    );

    const bridgeResult = await kit.bridge({
      from: { adapter, chain: "Arc_Testnet" },
      to: { adapter, chain: "Ethereum_Sepolia" },
      amount: swapResult.amountOut,
    });

    console.log("Bridge result:", inspect(bridgeResult, false, null, true));
  } catch (err) {
    console.log("ERROR", inspect(err, false, null, true));
  }
```

```
};
void swapAndBridge();
```

Customize your flow to **set slippage for the swap**, **collect bridge fees**, or **estimate costs before bridging**.

2.2. Run the script

Save the `index.ts` file and run the script in your terminal:

Shell

```
npx tsx --env-file=.env index.ts
```

2.3. Verify the transactions

After the script finishes, find the returned results in the terminal output:

- For the swap step, use the `explorerUrl` to verify the swap transaction on Arc Testnet.
- For the bridge step, use the `steps` array and each step's `explorerUrl` to verify the USDC bridge transfer on Arc Testnet and Ethereum Sepolia.

The following is an example of how the results might look in the terminal output. The values used are not real transactions.

Swap result Bridge result

```
{
  tokenIn: 'EURC',
  tokenOut: 'USDC',
  chain: { chain: 'Arc_Testnet', chainId: 5042002 },
  amountIn: '1.00',
  amountOut: '0.99',
  txHash: '0x78abb6a896e6b166925cae122b6ab2a6abd49ba23cbbd4749c99d5cccf205897'
  explorerUrl: 'https://testnet.arcscan.app/tx/0x78abb6a896e6b166925cae122b6ab
  fees: [ { token: 'EURC', amount: '0.001', type: 'provider' } ]
}
```

Was this page helpful? Yes No

How swap fees work SDK reference

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.